AI DZ

Academy

Maîtriser le Machine Learning

Un Cours Complet

Des Fondamentaux à l'Implémentation Avancée

Instructeurs:
Abdelhak Samir D. (PhD en Informatique et Automatique, Université de Lille)

Certification: À la fin du cours, les étudiants reçoivent le Certificat Professionnel en Machine Learning de l'AI DZ Academy

Prérequis: Programmation Python, calcul de base, algèbre linéaire

Durée: 4 semaines (12 heures de cours magistraux et travaux pratiques)

Contents

Vue d'Ensemble du Cours **3**

1 Le Paysage du Machine Learning **4**
1.1 Qu'est-ce que le Machine Learning? . . . 4
1.2 Une Brève Histoire du Machine Learning . . . 4
1.3 Où Utilisons-Nous le Machine Learning? . . . 4
1.3.1 Exemples Réels qui Aident les Gens . . . 4

2 Les Trois Principaux Types de Machine Learning **5**
2.1 Supervised Learning: Apprendre avec un Professeur . . . 5
2.1.1 Tâches Courantes en Supervised Learning . . . 6
2.2 Unsupervised Learning: Trouver des Motifs Cachés . . . 6
2.3 Reinforcement Learning: Apprendre par Essai et Erreur . . . 6

3 Approfondissement: Linear Regression **7**
3.1 Le modèle comme boîte noire: entrée → sortie . . . 7
3.2 Qu'y a-t-il à l'intérieur du modèle? . . . 8
3.3 Hypothèse: dataset linéaire . . . 8
3.4 Objectif de la linear regression . . . 8
3.5 Intuition . . . 9
3.6 Les Trois Étapes Magiques de l'Apprentissage . . . 9
3.7 1. Forward pass ("faire une prédiction") . . . 9
3.8 2. Calculer la loss ("à quel point c'était faux?") . . . 9
3.9 3. Backward pass ("apprendre de l'erreur") . . . 10
3.9.1 Mise à jour des paramètres (gradient descent) . . . 10
3.10 Tout assembler (une étape d'entraînement) . . . 10
3.11 Notes et intuition . . . 10

4 Maîtrise de PyTorch: Des Fondamentaux au Premier Modèle **11**
4.1 Qu'est-ce que PyTorch et Pourquoi le Choisir? . . . 11
4.2 Tensors: Le Cœur de PyTorch . . . 12
4.2.1 Que Sont les Tensors? . . . 12
4.2.2 Pourquoi Avons-Nous Besoin de Tensors en Machine Learning . . . 12
4.3 Génération de Données . . . 12
4.3.1 Implémentation du Code . . . 12
4.3.2 Explication . . . 13
4.3.3 Visualisation . . . 13
4.4 Division des Données . . . 13
4.4.1 Implémentation du Code . . . 13
4.4.2 Explication . . . 14
4.5 Définition du Modèle . . . 14
4.5.1 Implémentation du Code . . . 14
4.5.2 Explication . . . 15
4.6 Loss Function et Optimizer . . . 15
4.6.1 Implémentation du Code . . . 15
4.6.2 Explication . . . 16
4.7 Boucle d'Entraînement . . . 16

4.7.1 Implémentation du Code . . . 16
4.7.2 Explication des étapes d'entraînement . . . 16
4.8 Évaluation du Modèle . . . 17
4.8.1 Implémentation du Code . . . 17
4.8.2 Explication . . . 17
4.9 Faire des Prédictions . . . 18
4.9.1 Implémentation du Code . . . 18
4.9.2 Explication . . . 18
4.10 Résumé des Concepts Clés . . . 18

5 Problèmes Courants et Solutions 18

6 Non-linear regression — la même idée, plus les fonctions d'activation 19
6.1 Pourquoi ajouter des activation functions? . . . 19
6.2 Activation functions courantes (référence rapide) . . . 20
6.3 Exemple à une couche dans PyTorch (ReLU) . . . 20
6.4 Alternative: utiliser `tanh` ou `sigmoid` . . . 20
6.5 Intuition et quand utiliser . . . 21

7 Classification (vue d'ensemble) 21

8 Binary classification (détaillée) 21
8.1 Problème vs regression . . . 21
8.2 Loss function: binary cross-entropy (log loss) . . . 22
8.3 Intuition derrière le logarithme . . . 22
8.3.1 Exemples numériques concrets (pas à pas) . . . 22
8.4 Pourquoi cela correspond à la probabilité et au maximum de vraisemblance 23
8.5 Note pratique PyTorch . . . 23

9 Multiclass classification (bref) 24

10 Résumé rapide 24

11 Conclusion et Prochaines Étapes 24

Vue d'Ensemble du Cours

Bienvenue au Cours Phare de Machine Learning de l'AI DZ Academy!

Ce cours est conçu pour vous transformer d'un débutant à un praticien compétent en machine learning. Nous combinons des fondations mathématiques rigoureuses avec une implémentation pratique.

Objectifs d'Apprentissage

- **Comprendre** les fondations mathématiques des algorithmes de machine learning
- **Implémenter** des modèles de supervised, unsupervised et reinforcement learning
- **Maîtriser** PyTorch pour le deep learning et les réseaux de neurones
- **Développer** des compétences en prétraitement de données et feature engineering
- **Apprendre** l'évaluation de modèles, le tuning des hyperparamètres et les stratégies de déploiement
- **Construire** un portfolio de projets de machine learning

Structure du Cours

Semaine	Module	Sujets	Projets
1	Fondations	Types de ML, Révision Math, Bases Python	Configuration Environnement de Développement
2	Modèles Linéaires	Linear/Logistic Regression, Optimisation	Prédiction Prix des Maisons
3	Réseaux de Neurones	Perceptrons, Fonctions d'Activation, Backpropagation	Reconnaissance de Chiffres
4	Deep Learning	CNNs, RNNs, Transformers	Classification d'Images
5	Unsupervised Learning	Clustering, Réduction de Dimensionnalité	Segmentation de Clients
6	Sujets Avancés	Reinforcement Learning, GANs	Agent IA de Jeu
7	ML en Production	Déploiement de Modèles, MLOps, Éthique	Déployer Modèle sur le Cloud
8	Capstone	Projet ML de Bout en Bout	Présentation Projet Final

Table 1: Structure et Calendrier du Cours

1 Le Paysage du Machine Learning

1.1 Qu'est-ce que le Machine Learning?

Définition 1.1 (Machine Learning)**.** *Le Machine Learning (ML) c'est quand les ordinateurs apprennent à partir d'exemples et d'expériences, au lieu qu'on leur dise exactement quoi faire. C'est comme enseigner à un enfant en lui montrant beaucoup d'images de chats et de chiens jusqu'à ce qu'il puisse reconnaître la différence lui-même.*

1.2 Une Brève Histoire du Machine Learning

1.3 Où Utilisons-Nous le Machine Learning?

1.3.1 Exemples Réels qui Aident les Gens

Exemple 1.1 (Aider les Médecins)**.** ***Problème:*** *Il faut des années pour former des ophtalmologues à détecter les maladies oculaires*
Solution ML: *Montrer à un ordinateur des milliers de scans oculaires pour apprendre les motifs*
Résultat: *L'ordinateur peut aider les médecins à trouver des problèmes plus rapidement, surtout dans les villages isolés*

Exemple 1.2 (Protéger Votre Argent)**.** ***Problème:*** *Les banques ne peuvent pas vérifier chaque transaction pour fraude manuellement*

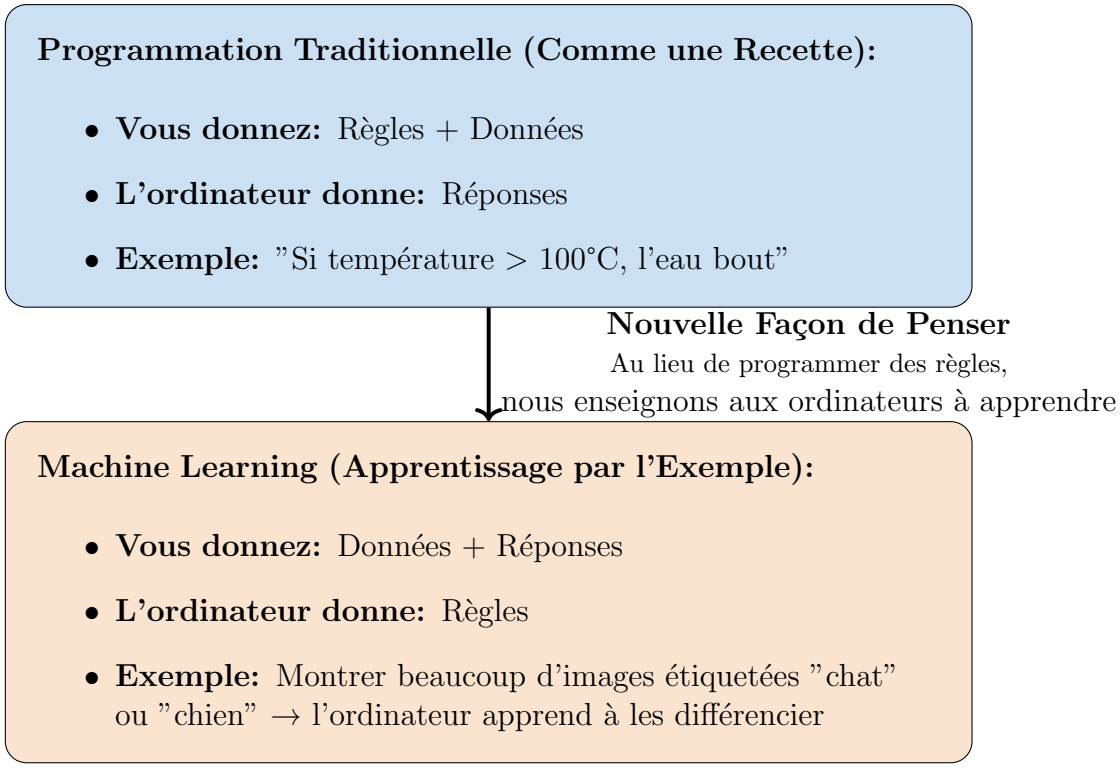


Figure 1: Différence Entre Programmation Traditionnelle et Machine Learning

Solution ML: *Enseigner à l'ordinateur à quoi ressemblent les transactions normales vs suspectes*
Résultat: *Votre carte est bloquée automatiquement si quelqu'un essaie de l'utiliser frauduleusement*

2 Les Trois Principaux Types de Machine Learning

2.1 Supervised Learning: Apprendre avec un Professeur

Définition 2.1 (Supervised Learning)**.** *Apprentissage avec des exemples étiquetés. Comme étudier avec des corrigés! Vous donnez à l'ordinateur des questions AVEC réponses, et il apprend à répondre à de nouvelles questions.*

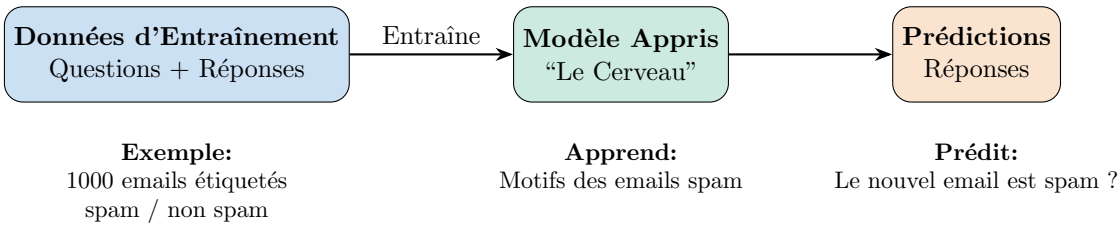


Figure 3: Processus de Supervised Learning

Décennie	Grande Idée	Ce que Cela a Rendu Possible
1950s	Première machine apprenante (Perceptron)	Reconnaissance simple de motifs, comme distinguer gauche de droite
1980s	Backpropagation (apprendre de ses erreurs)	Entraînement de réseaux plus intelligents avec plusieurs couches
1990s	Support Vector Machines	Meilleure classification, comme spam vs non-spam
2000s	Deep Learning	Ordinateurs qui peuvent "voir" (reconnaissance d'images) et "entendre" (reconnaissance vocale)
2010s	Big Data + GPUs Puissants	Apprentissage beaucoup plus rapide avec d'énormes quantités de données
2020s	Transformers (comme GPT)	Compréhension du langage incroyable, chatbots, traduction

Table 2: Le Machine Learning à Travers les Années

2.1.1 Tâches Courantes en Supervised Learning

2.2 Unsupervised Learning: Trouver des Motifs Cachés

Définition 2.2 (Unsupervised Learning)**.** *Apprentissage sans étiquettes. Comme explorer une nouvelle ville sans carte! L'ordinateur trouve des motifs et des groupes dans les données tout seul.*

Exemple 2.1 (Organiser Votre Musique)**.** ***Problème:*** *Vous avez 5,000 chansons sans playlist*
Solution ML: *L'ordinateur écoute toutes les chansons et les groupe par similarité*
Résultat: *Crée des playlists comme "Chill Lo-Fi", "Mix Entraînement", "Focus Étude" automatiquement*

2.3 Reinforcement Learning: Apprendre par Essai et Erreur

Définition 2.3 (Reinforcement Learning)**.** *Apprentissage en essayant différentes actions et en voyant ce qui fonctionne le mieux. Comme apprendre à faire du vélo - vous essayez, parfois vous tombez (feedback négatif), parfois vous gardez l'équilibre (feedback positif), et vous vous améliorez progressivement.*

Exemple 2.2 (Entraîner une IA de Jeu)**.** ***Problème:*** *Comment rendre l'IA bonne à un jeu vidéo?*
Approche RL: *Laisser l'IA jouer des millions de fois, récompensant les bons coups et pénalisant les mauvais*
Résultat: *L'IA devient surhumaine à des jeux comme Go, Dota 2, ou StarCraft*

Prédire des Nombres (Regression)	**Choisir des Catégories (Classification)**	**Sorties Complexes**
• **Objectif:** Prédire un nombre • **Exemples:** – Prix d'une maison – Température demain – Votre score de test	• **Objectif:** Choisir une catégorie • **Exemples:** – Spam ou pas spam? – Chat ou chien? – Visage heureux ou triste?	• **Objectif:** Générer sortie complexe • **Exemples:** – Écrire une phrase – Dessiner une image – Traduire des langues

Table 3: Différents Types de Problèmes de Supervised Learning

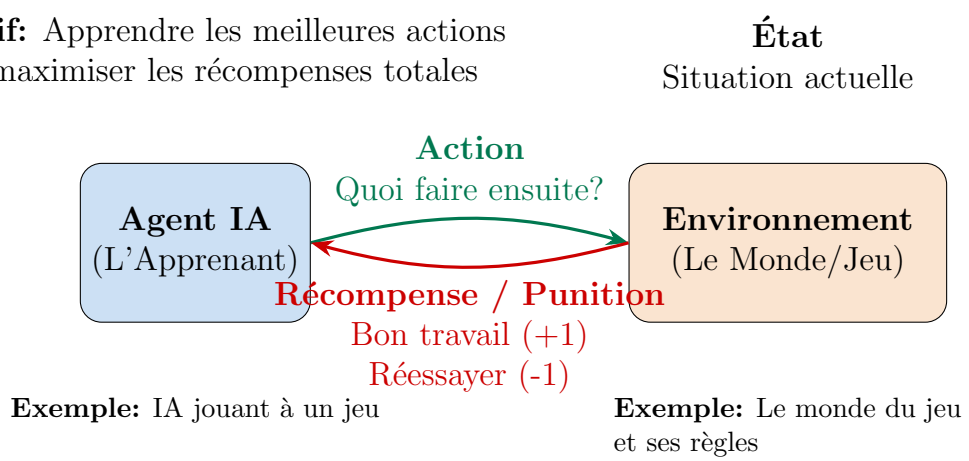


Figure 4: Reinforcement Learning: La Boucle de Feedback

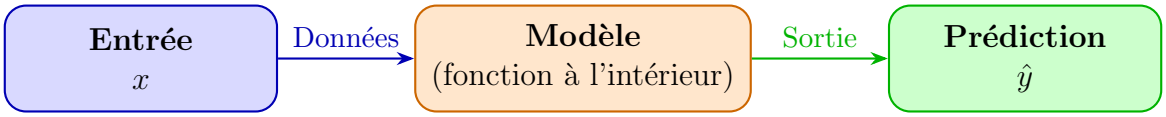
3 Approfondissement: Linear Regression

Avant d'approfondir l'entraînement, comprenons clairement ce qu'est un **modèle**.

3.1 Le modèle comme boîte noire: entrée → sortie

Au plus haut niveau, un modèle peut être vu comme un système simple:

- Il reçoit une **entrée** x
- Il produit une **sortie** $\hat{y}$



3.2 Qu'y a-t-il à l'intérieur du modèle?

De manière très simple, **il n'y a rien de magique à l'intérieur du modèle**. Un modèle est juste une **fonction mathématique**:

$$\hat{y} = f(x)$$

- L'entrée est x
- La sortie est $\hat{y}$
- La fonction $f(\cdot)$ est choisie en fonction du **type de dataset**

Idée importante: Différents datasets $\Rightarrow$ différents types de fonctions.

3.3 Hypothèse: dataset linéaire

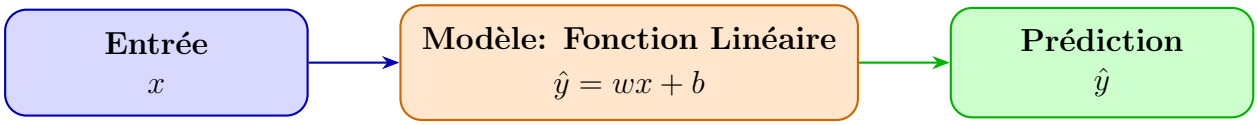
Dans cette première section, nous supposons que notre dataset est **linéaire**. Cela signifie que la relation entre l'entrée x et la sortie y peut être approximée par une ligne droite.

Nous choisissons donc une **fonction linéaire** pour notre modèle:

$$\hat{y} = wx + b$$

- w est le **weight** (pente de la ligne)
- b est le **bias** (décalage vertical)

Maintenant notre modèle n'est plus un mystère. À l'intérieur de la boîte, nous avons simplement:



3.4 Objectif de la linear regression

Notre dataset contient plusieurs points (x, y). Le modèle produit des prédictions $(x, \hat{y})$.

L'objectif principal de la linear regression est:

> Trouver les meilleures valeurs de w et b telles que la ligne $\hat{y} = wx + b$ s'ajuste au dataset aussi bien que possible.

En pratique, cela signifie:

- Les valeurs prédites $\hat{y}$ doivent être proches des valeurs réelles y
- La distance (erreur) entre la ligne et les points de données doit être aussi petite que possible

Cette distance est mesurée à l'aide d'une **loss function**, et l'entraînement consiste à ajuster w et b pour **minimiser cette distance**.

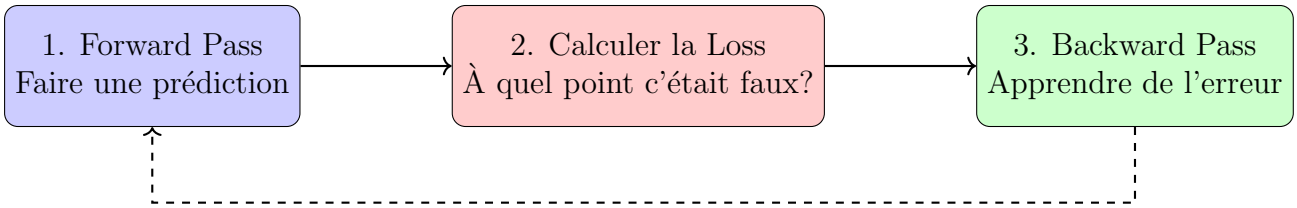
3.5 Intuition

- Le modèle commence avec des valeurs aléatoires de w et b
- La ligne est généralement mal ajustée au début
- Après de nombreuses étapes d'apprentissage, la ligne se déplace et pivote lentement
- Finalement, elle s'ajuste aux données aussi étroitement que possible

Point clé: La linear regression consiste simplement à trouver la meilleure ligne droite qui explique la relation entre les entrées et les sorties en minimisant la distance entre les prédictions et les points de données réels.

3.6 Les Trois Étapes Magiques de l'Apprentissage

Chaque fois que votre modèle apprend, il passe par trois étapes:



Ci-dessous, nous expliquons chaque étape de manière simple et concrète avec un petit exemple mathématique.

3.7 1. Forward pass ("faire une prédiction")

Le modèle utilise ses paramètres actuels (weights et biases, écrits collectivement comme θ) pour produire une prédiction pour une entrée x. Nous écrivons le modèle comme une fonction $\hat{y} = f_\theta(x)$. **Exemple (très simple):** un modèle linéaire avec weight W et bias b: $\hat{y} = Wx + b$. *Intuition:* le forward pass consiste simplement à faire passer votre entrée à travers les équations du modèle pour voir ce qu'il prédit.

3.8 2. Calculer la loss ("à quel point c'était faux?")

Nous comparons la prédiction du modèle $\hat{y}$ à la cible réelle y en utilisant une loss function $\mathcal{L}(\hat{y}, y)$. Un choix courant pour la regression est l'erreur quadratique moyenne (MSE) pour un seul échantillon: $\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2$. *Intuition:* la loss est un nombre unique qui nous dit à quel point la prédiction était mauvaise — plus grand signifie pire.

Exemple numérique concret (un échantillon). Supposons $x = 2$, $y = 3$, et paramètres initiaux $W = 0.5$, $b = 0$.

$$\text{Prédiction: } \hat{y} = Wx + b = 0.5 \times 2 + 0 = 1.$$
$$\text{Loss: } \mathcal{L} = (\hat{y} - y)^2 = (1 - 3)^2 = 4.$$

Donc la prédiction du modèle est loin de la valeur réelle et la loss égale 4.

3.9 3. Backward pass ("apprendre de l'erreur")

Le backward pass calcule comment modifier les paramètres θ pour réduire la loss. Concrètement, nous calculons le gradient (dérivées partielles) de la loss par rapport à chaque paramètre: $\nabla_\theta \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \dots\right)$. Pour le modèle linéaire simple $\hat{y} = Wx + b$ et loss $\mathcal{L} = (\hat{y} - y)^2$, les dérivées pour un échantillon sont:

$$\frac{\partial \mathcal{L}}{\partial W} = 2(\hat{y} - y)\,\frac{\partial \hat{y}}{\partial W} = 2(\hat{y} - y)\,x,$$
$$\frac{\partial \mathcal{L}}{\partial b} = 2(\hat{y} - y)\,\frac{\partial \hat{y}}{\partial b} = 2(\hat{y} - y).$$

Continuer l'exemple numérique. En utilisant les nombres ci-dessus: $\hat{y} - y = 1 - 3 = -2$. Donc $\frac{\partial \mathcal{L}}{\partial W} = 2(-2) \cdot 2 = -8$, $\frac{\partial \mathcal{L}}{\partial b} = 2(-2) = -4$. Ces gradients nous indiquent dans quelle direction modifier chaque paramètre pour réduire la loss.

3.9.1 Mise à jour des paramètres (gradient descent)

Une règle de mise à jour simple est le gradient descent: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$, où $\eta > 0$ est le learning rate (une petite taille de pas). En utilisant $\eta = 0.1$ dans notre exemple: $W \leftarrow 0.5 - 0.1 \times (-8) = 0.5 + 0.8 = 1.3$, $b \leftarrow 0 - 0.1 \times (-4) = 0.4$. Après cette mise à jour, le modèle prédira une valeur plus proche de $y = 3$ pour la même entrée $x = 2$.

3.10 Tout assembler (une étape d'entraînement)

1. **Forward:** calculer $\hat{y} = f_\theta(x)$.
2. **Loss:** calculer $\mathcal{L}(\hat{y}, y)$.
3. **Backward:** calculer les gradients $\nabla_\theta \mathcal{L}$.
4. **Update:** $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$.

3.11 Notes et intuition

- **Pourquoi les gradients?** Les gradients pointent dans la direction de la plus grande augmentation de la loss. Soustraire un petit multiple du gradient déplace les paramètres dans la direction qui réduit la loss.
- **Batch vs échantillon unique:** En pratique, nous calculons généralement la loss moyenne et les gradients moyens sur un petit ensemble d'exemples (un *mini-batch*) plutôt qu'un seul échantillon. Cela lisse les mises à jour et aide souvent l'entraînement.
- **Learning rate η:** trop grand → entraînement instable; trop petit → apprentissage très lent. C'est un hyperparamètre que vous ajustez.
- **Backpropagation:** pour les réseaux de neurones profonds, le backward pass utilise la règle de la chaîne de manière répétée pour calculer les gradients couche par couche. Les frameworks (PyTorch, TensorFlow) calculent ces gradients automatiquement.

Pensée finale: Pensez au modèle comme un étudiant passant un quiz. Le forward pass est l'étudiant répondant à la question, la loss est le professeur notant à quel point la réponse est fausse, et le backward pass est l'étudiant étudiant le feedback et ajustant sa façon de penser pour que la prochaine réponse soit meilleure.

4 Maîtrise de PyTorch: Des Fondamentaux au Premier Modèle

4.1 Qu'est-ce que PyTorch et Pourquoi le Choisir?

Définition 4.1 (PyTorch)**.** *PyTorch est un framework de machine learning open-source qui fournit deux fonctionnalités principales:*

- ***Calcul tensoriel*** *avec forte accélération GPU*
- ***Réseaux de neurones profonds*** *construits sur un système autograd basé sur bande*

Pourquoi PyTorch est adapté aux débutants:

1. **Nature pythonique**: Se sent comme écrire du code Python régulier
2. **Graphe de calcul dynamique**: Construire et modifier les réseaux à la volée
3. **Débogage facile**: Utiliser les outils de débogage Python standard
4. **Communauté forte**: Tutoriels et documentation étendus
5. **Adapté à la recherche**: Largement utilisé dans le milieu académique et l'industrie

```
# Comparaison avec TensorFlow (simplifie)
# PyTorch - Plus intuitif, pythonique
model(x)  # Appeler simplement le modele comme une fonction

# TensorFlow - Plus declaratif
model.predict(x)  # Methode speciale necessaire
```

Listing 1: Pourquoi PyTorch plutôt que d'autres frameworks

4.2 Tensors: Le Cœur de PyTorch

4.2.1 Que Sont les Tensors?

Analogie

Pensez aux tensors comme des **conteneurs pour les données**:

- **Scalaire** (tensor 0D): Un seul nombre → Température: 25°C
- **Vecteur** (tensor 1D): Liste de nombres → Températures quotidiennes: [25, 26, 24]
- **Matrice** (tensor 2D): Table de nombres → Grille de températures: [[25, 26], [24, 25]]
- **Tensor 3D+**: Dimensions supérieures → Données vidéo: [frames × hauteur × largeur × couleurs]

4.2.2 Pourquoi Avons-Nous Besoin de Tensors en Machine Learning

- **Calcul efficace**: Opérations spécialisées optimisées pour la vitesse
- **Accélération GPU**: Déplacer les calculs vers le GPU pour une accélération de 50–100×
- **Différentiation automatique**: Suivre les opérations pour le calcul du gradient
- **Format standardisé**: Toutes les bibliothèques ML comprennent le format tensor

4.3 Génération de Données

4.3.1 Implémentation du Code

```
import torch
import numpy as np
import matplotlib.pyplot as plt

# Definir les vrais parametres (inconnus du modele initialement)
a = 0.3  # Vraie pente
b = 0.7  # Vrai intercept

# Creer les donnees d'entree: 50 points de 0 a 0.98
X = torch.arange(0., 1., 0.02).float()  # shape (50,)

# Generer les valeurs cibles avec relation lineaire (pas de bruit ici)
y = a * X + b

print(f"X = {X}\n\ny = {y}")
```

Listing 2: Generation de donnees synthetiques

4.3.2 Explication

Nous créons des données synthétiques suivant une relation linéaire $y = 0.3x + 0.7$. Cette approche nous permet de connaître la "vérité terrain" et d'évaluer précisément les performances de notre modèle.

4.3.3 Visualisation

```
plt.scatter(X.numpy(), y.numpy())
plt.xlabel('X')
plt.ylabel('y')
plt.title('Donnees Lineaires Synthetiques')
plt.show()
```

Listing 3: Visualisation des donnees

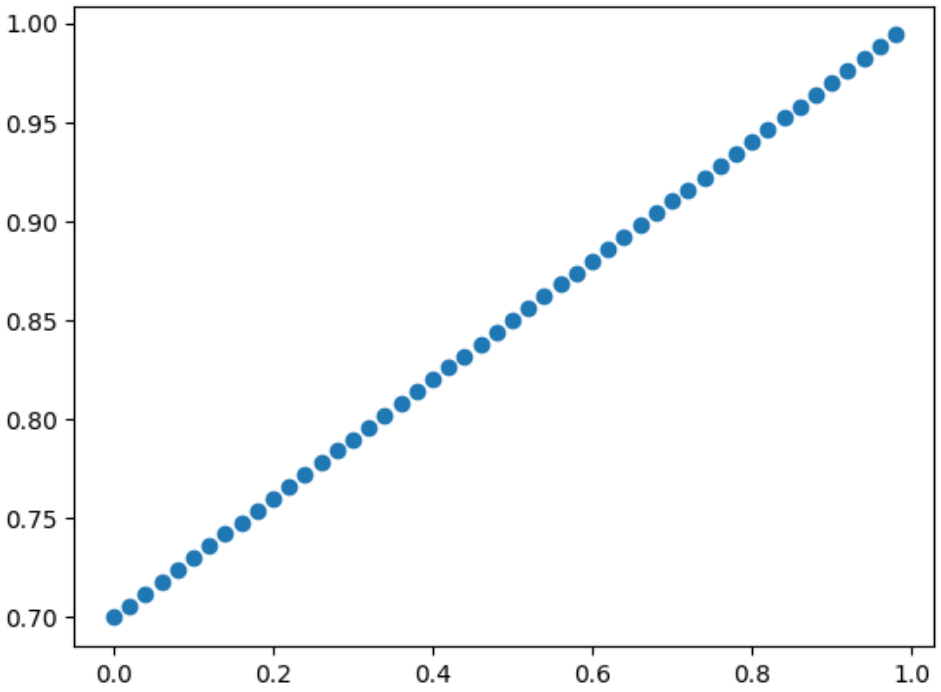


Figure 5: Représentation visuelle de nos données linéaires synthétiques

4.4 Division des Données

4.4.1 Implémentation du Code

```
split = int(0.8 * len(X))
X_train, y_train = X[:split], y[:split]
X_test, y_test = X[split:], y[split:]

plt.scatter(X_train.numpy(), y_train.numpy(), label='Donnees d\'
    entrainement')
plt.scatter(X_test.numpy(), y_test.numpy(), c='red', label='Donnees de
    test')
plt.xlabel('X')
```

```
plt.ylabel('y')
plt.title('Division Donnees Entrainement vs Test')
plt.legend()
plt.show()
```

Listing 4: Division des donnees en ensembles d'entrainement et de test

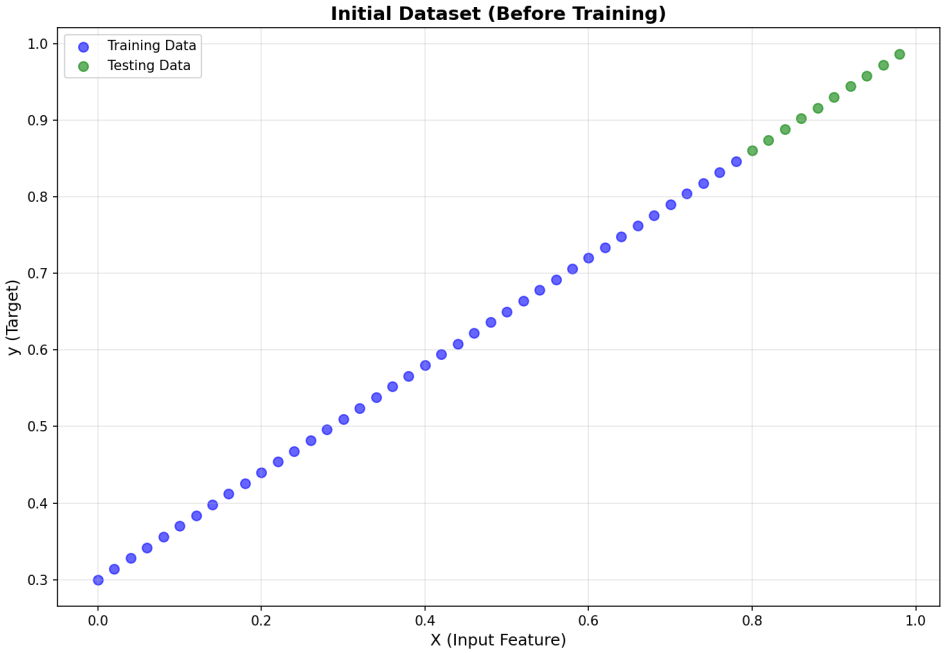


Figure 6: Représentation visuelle de nos datasets d'entraînement et de test

4.4.2 Explication

Nous divisons nos données en ensembles d'entraînement (40 points) et de test (10 points). Cela nous permet de:

- Entraîner le modèle sur l'ensemble d'entraînement
- Évaluer ses performances sur des données de test non vues
- Vérifier l'overfitting

4.5 Définition du Modèle

4.5.1 Implémentation du Code

```
import torch.nn as nn
torch.manual_seed(43)  # Pour la reproductibilite

class LinearRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        # Initialiser les parametres (weight et bias uniques)
        # Utiliser nn.Parameter pour qu'ils soient enregistres et
   optimises
```

```
        self.w = nn.Parameter(torch.randn(1, dtype=torch.float32))
        self.b = nn.Parameter(torch.randn(1, dtype=torch.float32))

    def forward(self, x):
        # x est cense etre un tensor 1-D (shape: [N,])
        return self.w * x + self.b

model = LinearRegressionModel()
```

Listing 5: Definition du modele de linear regression

4.5.2 Explication

- `nn.Module`: Classe de base pour tous les modules de réseau de neurones.
- `nn.Parameter`: Tensors qui seront optimisés pendant l'entraînement.
- `requires_grad`: par défaut pour `nn.Parameter` est True, donc les gradients seront calculés.
- `forward()`: Définit le calcul effectué à chaque appel.
- **Note**: Aucune activation (ReLU) n'est utilisée ici car nous voulons un vrai modèle linéaire.

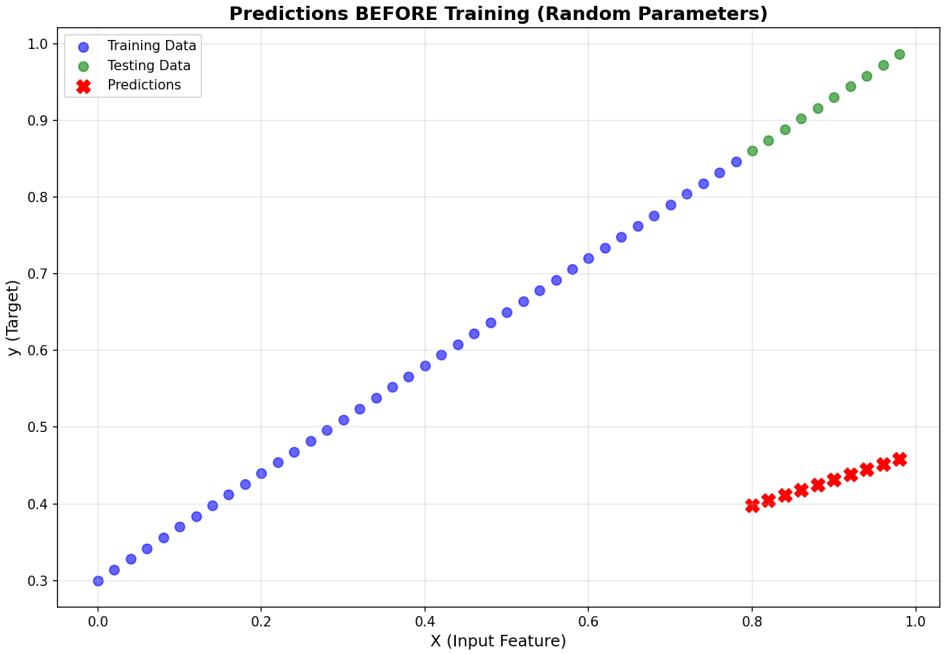


Figure 7: Représentation visuelle de la prédiction du modèle avant l'entraînement

4.6 Loss Function et Optimizer

4.6.1 Implémentation du Code

```
criterion = nn.L1Loss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Listing 6: Configuration de la loss function et de l'optimizer

4.6.2 Explication

La loss function mesure l'écart entre les prédictions et les cibles. L'optimizer met à jour les paramètres du modèle en utilisant les gradients calculés à partir de la loss. Pour les mises à jour de type gradient-descent: $\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta)$ où η est le learning rate.

4.7 Boucle d'Entraînement

4.7.1 Implémentation du Code

```
epochs = 250
loss_history = []

for epoch in range(epochs):
    model.train()  # Mettre le modele en mode entrainement

    # Forward pass: calculer les predictions sur l'ensemble d'
    entrainement
    y_pred = model(X_train)

    # Calculer la loss
    loss = criterion(y_pred, y_train)
    loss_history.append(loss.item())

    # Backward pass et optimisation
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# Tracer la courbe de loss
import matplotlib.pyplot as plt
plt.plot(loss_history)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Loss d\'Entrainement au Fil des Epochs')
plt.show()
```

Listing 7: Entrainement du modele

4.7.2 Explication des étapes d'entraînement

1. **Forward Pass**: Calculer les prédictions $\hat{y} = f_\theta(X)$.
2. **Calcul de la Loss**: Calculer l'erreur entre les prédictions et les valeurs réelles.
3. **Backward Pass**: Calculer les gradients $\nabla_\theta J$ en utilisant la backpropagation.
4. **Mise à Jour des Paramètres**: Ajuster les paramètres dans la direction qui minimise la loss.

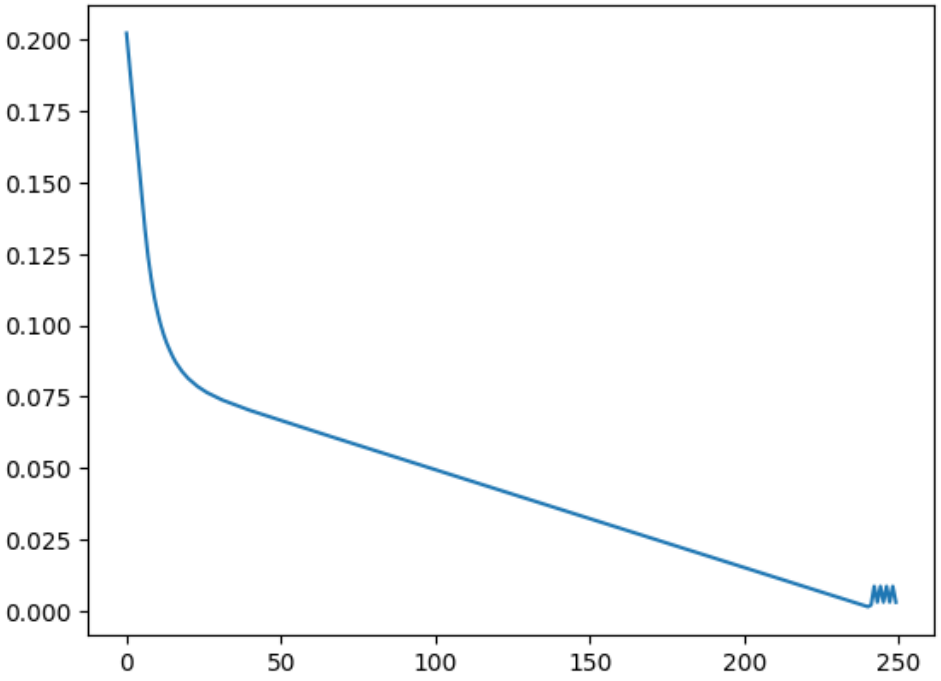


Figure 8: Loss d'Entraînement au Fil des Epochs

4.8 Évaluation du Modèle

4.8.1 Implémentation du Code

```
model.eval()  # Mettre le modele en mode evaluation
with torch.no_grad():  # Desactiver le calcul du gradient pour l'
    inference
     y_test_pred = model(X_test)

plt.scatter(X_train.numpy(), y_train.numpy(), label='Donnees d\'
    entrainement')
plt.scatter(X_test.numpy(), y_test.numpy(), c='green', label='Vraies
    valeurs de test')
plt.scatter(X_test.numpy(), y_test_pred.numpy(), c='red', label='
    Predictions')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Predictions du Modele vs Valeurs Reelles')
plt.legend()
plt.show()
```

Listing 8: Evaluation sur l'ensemble de test

4.8.2 Explication

- `model.eval()`: Met le modèle en mode évaluation (affecte les couches comme dropout ou batchnorm).
- `torch.no_grad()`: Désactive le calcul du gradient pour économiser de la mémoire pendant l'inférence.

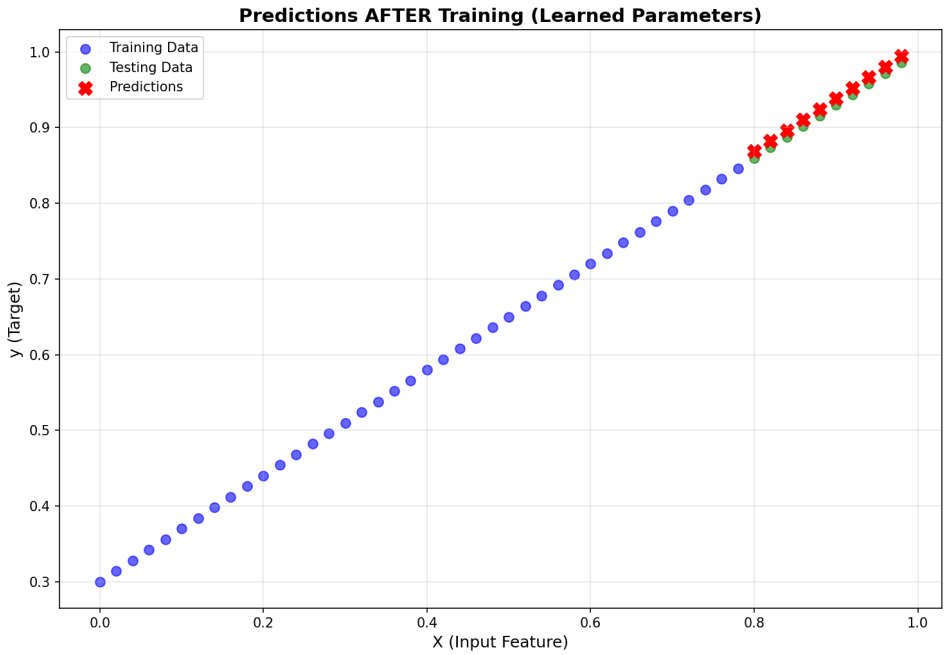


Figure 9: Prédictions du Modèle après entraînement vs Valeurs Réelles

- Le graphique montre à quel point le modèle généralise aux données non vues.

4.9 Faire des Prédictions

4.9.1 Implémentation du Code

```
x = 0.79
y_true = a * x + b  # Valeur reelle utilisant les parametres connus
y_pred_single = model(torch.tensor([x], dtype=torch.float32))

print(f"Valeur reelle: y = {y_true:.3f}")
print(f"Valeur predite: y_hat = {y_pred_single.item():.3f}")
```

Listing 9: Faire une prediction unique

4.9.2 Explication

Pour $x = 0.79$:

- Valeur réelle: $y = 0.3 \times 0.79 + 0.7 \approx 0.937$
- Prédiction du modèle: montrée par `y_pred_single`
- Une petite différence indique un apprentissage réussi

4.10 Résumé des Concepts Clés

5 Problèmes Courants et Solutions

- **Learning rate**: Trop élevé cause la divergence; trop faible cause une convergence lente.

Concept	Explication
Parameters (w, b)	Weight et bias appris du modèle linéaire
Forward Pass	Calcul des prédictions à partir des entrées
Loss Function	Mesure l'erreur entre prédictions et valeurs réelles
Backward Pass	Calcul des gradients via backpropagation
Optimizer	Met à jour les paramètres pour minimiser la loss
Training Loop	Étapes itératives de forward, loss, backward et optimisation
Model Evaluation	Test sur données non vues pour évaluer la généralisation

Table 4: Concepts clés dans la linear regression avec PyTorch

- **Normalisation des données**: Non nécessaire pour cet exemple synthétique mais important pour les datasets du monde réel.
- **Accumulation de gradient**: Toujours appeler `optimizer.zero_grad()` avant `loss.backward()`.

6 Non-linear regression — la même idée, plus les fonctions d'activation

Un modèle de **non-linear regression** est conceptuellement le même que la linear regression, mais nous insérons des *activation functions* (fonctions non linéaires) dans le modèle. Là où la linear regression utilise une seule transformation linéaire $\hat{y} = wx + b$, un modèle de non-linear regression applique une fonction non linéaire $\sigma(\cdot)$ à la combinaison linéaire: $\hat{y} = \sigma(wx + b)$. Parce que σ est non linéaire, la transformation de x à $\hat{y}$ peut représenter des relations beaucoup plus riches qu'une ligne droite.

6.1 Pourquoi ajouter des activation functions?

- **Introduire la non-linéarité:** Sans activation, un empilement de couches linéaires équivaut toujours à une seule fonction linéaire. Les activations permettent au modèle d'apprendre des formes courbes, par morceaux ou plus complexes.
- **Expressivité du modèle:** Avec des activations appropriées, même de petits réseaux peuvent approximer des fonctions compliquées (c'est l'intuition derrière les résultats d'approximation universelle).
- **Contrôler la plage de sortie:** Certaines activations (par ex. tanh, sigmoid) limitent la plage de sortie, ce qui peut être utile selon la variable cible.

6.2 Activation functions courantes (référence rapide)

ReLU: $\mathrm{ReLU}(z) = \max(0, z)$ (simple, rapide, linéaire par morceaux)

Leaky ReLU: $\mathrm{LeakyReLU}(z) = \begin{cases} z & z \geq 0 \\ \alpha z & z < 0 \end{cases} \quad (\alpha > 0)$

Sigmoid: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$ (mappe vers $(0, 1)$)

Tanh: $\tanh(z) = \dfrac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$ (mappe vers $(-1, 1)$)

Note importante pour la regression: si votre cible y est non bornée et à valeur réelle, vous ne forcez généralement *pas* la couche finale à travers une activation bornée comme sigmoid — au lieu de cela, vous utilisez des activations non linéaires dans les couches cachées et gardez la sortie finale linéaire (identité). Dans l'exemple simple à une couche ci-dessous, nous montrons l'activation appliquée à la sortie linéaire pour illustrer l'idée; pour de nombreuses tâches de regression, vous pourriez préférer $\hat{y} = wx + b$ comme étape finale et placer des activations uniquement si vous ajoutez des couches cachées.

6.3 Exemple à une couche dans PyTorch (ReLU)

Ci-dessous se trouve un exemple PyTorch corrigé et fonctionnel qui montre comment ajouter une activation à un modèle linéaire simple.

```python
import torch
import torch.nn as nn

torch.manual_seed(43)

class NonLinearRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        # weight et bias uniques (scalaire)
        self.w = nn.Parameter(torch.randn(1, dtype=torch.float32))
        self.b = nn.Parameter(torch.randn(1, dtype=torch.float32))
        # activation (non-linearite)
        self.relu = nn.ReLU()

    def forward(self, x):
        # s'assurer que x est un tensor float; broadcasting fonctionne pour w, b scalaires
        return self.relu(self.w * x + self.b)

# exemple d'utilisation
model = NonLinearRegressionModel()
x = torch.tensor([0.1, 0.5, 1.0], dtype=torch.float32)
y_hat = model(x)  # y_hat est non-lineairement lie a x
print(y_hat)
```

Listing 10: Modele a une couche avec activation ReLU

6.4 Alternative: utiliser `tanh` ou `sigmoid`

Si vous voulez une transformation non linéaire lisse (non linéaire par morceaux), vous pouvez remplacer `nn.ReLU()` par `nn.Tanh()` ou `nn.Sigmoid()`:

```
# remplacer l'activation dans le modele:
self.activation = nn.Tanh()        # ou nn.Sigmoid(), nn.LeakyReLU
    (0.01), ...
# forward: return self.activation(self.w * x + self.b)
```

Listing 11: Changement de fonction d'activation

6.5 Intuition et quand utiliser

- Si les données sont approximativement linéaires: un modèle linéaire simple peut suffire et est plus facile à interpréter.
- Si les données montrent des courbes, saturation ou comportement par morceaux: ajouter des non-linéarités (activations) et/ou des couches cachées.
- Pour des tâches simples, une seule couche linéaire + activation peut déjà modéliser des relations linéaires par morceaux (ReLU) — mais des réseaux plus profonds ou plus larges augmentent la flexibilité.
- Pour la regression vers des cibles réelles non bornées, préférer **aucune activation de limitation finale** (garder la sortie finale linéaire) et placer les activations dans les couches cachées.

7 Classification (vue d'ensemble)

La classification est la tâche d'assigner des étiquettes discrètes aux entrées. Il existe deux familles courantes:

- **Binary classification:** seulement deux classes possibles (souvent notées 0 et 1). Exemple: spam vs non-spam.
- **Multiclass classification:** plus de deux classes et chaque exemple appartient à exactement une classe (single-label). Exemple: reconnaissance de chiffres (0–9).

La principale différence: dans la binary classification, nous prédisons une seule probabilité ("est-ce la classe 1?"); dans multiclass, nous prédisons une distribution de probabilité sur $K > 2$ classes.

8 Binary classification (détaillée)

8.1 Problème vs regression

Si vous essayez d'utiliser un modèle de linear regression simple pour un problème binaire, vous obtenez des sorties à valeurs réelles qui ne sont pas des probabilités. Nous avons besoin de sorties dans l'intervalle $[0, 1]$ qui peuvent être interprétées comme $\Pr(\text{classe} = 1 \mid x)$. Pour obtenir des probabilités, nous appliquons une activation **sigmoid** à la sortie linéaire (logit).

$z = wx + b$ (logit)

$\hat{p} \equiv \hat{y} = \sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$

Maintenant $\hat{y}$ est une probabilité: grand z positif $\Rightarrow \hat{y} \approx 1$; grand z négatif $\Rightarrow \hat{y} \approx 0$.

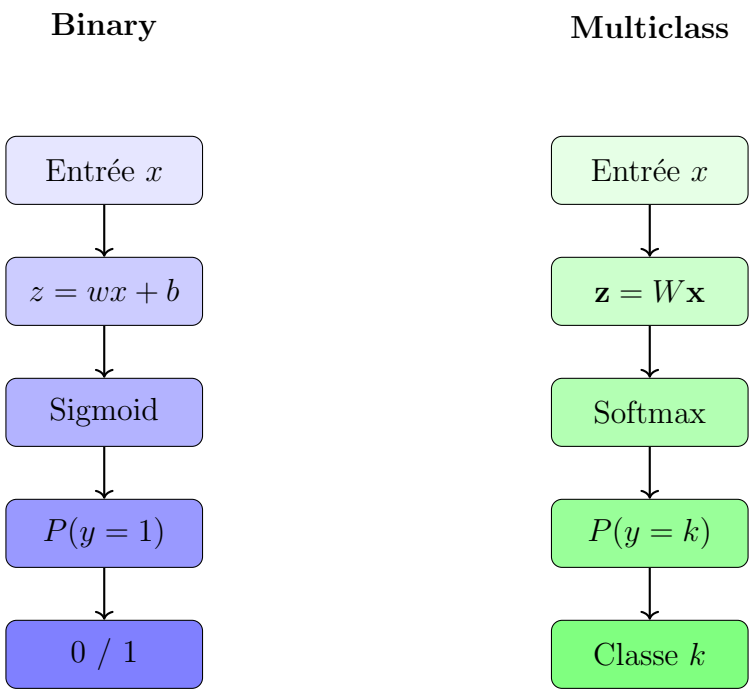


Figure 10: Binary vs multiclass classification (vue simplifiée)

8.2 Loss function: binary cross-entropy (log loss)

Pour un seul échantillon avec étiquette réelle $y \in \{0, 1\}$ et probabilité prédite $\hat{y} \in (0, 1)$, la binary cross-entropy loss (aussi appelée log loss) est

$$J(\hat{y}, y) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right].$$

Cette formule fait deux choses en une seule expression:

- Si $y = 1$, la loss se réduit à $J = -\log(\hat{y})$.
- Si $y = 0$, la loss se réduit à $J = -\log(1 - \hat{y})$.

8.3 Intuition derrière le logarithme

- Le logarithme fait que la loss *pénalise très fortement les prédictions fausses confiantes*. Parce que $\log(z) \to -\infty$ quand $z \to 0^+$, si le modèle assigne une probabilité proche de 0 à la vraie classe, la loss devient très grande.
- Inversement, si le modèle assigne une probabilité proche de 1 à la vraie classe, $-\log(\hat{y})$ est petit, donc la loss est petite.

8.3.1 Exemples numériques concrets (pas à pas)

Cas A: étiquette réelle $y = 1$, bonne prédiction $\hat{y} = 0.99$. $J = -\log(0.99)$

Calcul (logarithme naturel): $\log(0.99) \approx -0.0100503 \quad \Rightarrow \quad J \approx 0.01005$

Donc une prédiction correcte confiante donne une loss très petite.

Cas B: étiquette réelle $y = 1$, mauvaise prédiction $\hat{y} = 0.01$. $J = -\log(0.01)$

$\log(0.01) = \log(10^{-2}) = -2\log(10) \approx -4.60517 \quad \Rightarrow \quad J \approx 4.60517$

Donc une prédiction fausse confiante produit une loss très grande.

Symétrie pour $y = 0$. Si $y = 0$, la loss est $J = -\log(1 - \hat{y})$.

- Si $\hat{y} = 0.01$: $J = -\log(0.99) \approx 0.01005$ (petite loss)
- Si $\hat{y} = 0.99$: $J = -\log(0.01) \approx 4.60517$ (grande loss)

Cette symétrie explique pourquoi une seule formule couvre les deux cas.

8.4 Pourquoi cela correspond à la probabilité et au maximum de vraisemblance

La binary cross-entropy est (à une échelle près) la log-vraisemblance négative d'un modèle de Bernoulli. Minimiser la moyenne de cette loss sur le dataset est équivalent à l'estimation par maximum de vraisemblance.

8.5 Note pratique PyTorch

Pour la stabilité numérique, vous devriez généralement alimenter la sortie linéaire brute (logits) dans `nn.BCEWithLogitsLoss()` plutôt que d'appliquer `sigmoid` vous-même et d'utiliser `nn.BCELoss()`. `BCEWithLogitsLoss` combine les deux de manière stable.

```python
import torch
import torch.nn as nn

torch.manual_seed(43)

class BinaryLinearModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.w = nn.Parameter(torch.randn(1))
        self.b = nn.Parameter(torch.randn(1))

    def forward(self, x):
        return self.w * x + self.b

model = BinaryLinearModel()
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# boucle d'entrainement (esquisse)
logits = model(X)
loss = criterion(logits, y)  # y dans {0.0, 1.0}
loss.backward()
optimizer.step()
```

Listing 12: Modele binaire avec logits et loss stable dans PyTorch

Pour l'inférence: $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$

Ensuite convertir la probabilité en classe en utilisant un seuil (typiquement 0.5): $\hat{c} = \begin{cases} 1 & \text{si } \hat{y} \geq 0.5 \\ 0 & \text{si } \hat{y} < 0.5 \end{cases}$

9 Multiclass classification (bref)

- Pour la multiclass classification single-label, nous utilisons une couche **softmax**: $\hat{p}_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \ldots, K$
- La loss est la categorical cross-entropy (PyTorch: `nn.CrossEntropyLoss`). Elle attend des logits bruts et des étiquettes entières $0, \ldots, K-1$.

10 Résumé rapide

- **Binary classification:** sigmoid + binary cross-entropy (`BCEWithLogitsLoss`)
- **Multiclass classification:** softmax + categorical cross-entropy (`CrossEntropyLoss`)
- **Intuition du log:** petite loss quand confiant et correct, énorme loss quand confiant et faux

11 Conclusion et Prochaines Étapes

Félicitations!

Vous avez maîtrisé le premier module fondamental
du Cours de Machine Learning de l'AI DZ Academy.

Vous avez maintenant une compréhension solide de:
Linear Regression & Fondamentaux de PyTorch

Module Suivant:

Réseaux de Neurones & Deep Learning

- **Perceptrons**
- **Activations**
- **Backpropagation**
- **CNNs**
- **RNNs**

Prêt à construire des systèmes intelligents? Le voyage continue...

Ressources Supplémentaires

Livres:

- *Pattern Recognition and Machine Learning* - Christopher Bishop
- *Deep Learning* - Ian Goodfellow, Yoshua Bengio, Aaron Courville
- *Hands-On Machine Learning* - Aurélien Géron

Cours en Ligne:

- Stanford CS229: Machine Learning (Andrew Ng)
- Fast.ai: Practical Deep Learning for Coders
- Spécialisations DeepLearning.AI sur Coursera

Articles de Recherche:

- "Attention Is All You Need" - Vaswani et al. (2017)
- "ImageNet Classification with Deep CNNs" - Krizhevsky et al. (2012)
- "Playing Atari with Deep RL" - Mnih et al. (2013)

Outils Pratiques:

- **MLflow**: Gestion du Cycle de Vie du Machine Learning
- **DVC**: Contrôle de Version des Données
- **Weights & Biases**: Suivi d'Expériences
- **Gradio**: Interface de Déploiement de Modèles

AI DZ ACADEMY

Responsabiliser la Prochaine Génération d'Ingénieurs IA

Contact: aidzacademy@gmail.com — whatsapp: +213658597060

Instagram:/ai_dz_academy/

Site Web: www.aidzacademy.org

Alger, Algérie — © 2025 AI DZ Academy